FACULTY OF ENGINEERING AND BASIC SCIENCES
ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE

COURSE: ETL (G01)
Workshop-3: Streaming ETL with Apache Kafka and Machine Learning

1. Introduction

This workshop focuses on the transition from traditional batch ETL pipelines to event-driven streaming pipelines.

Students have already worked with:

- ETL pipelines in Python
- Data cleaning and transformation
- Data quality concepts
- Dimensional modeling
- Apache Airflow
- Batch orchestration
- Data Warehouses

This workshop introduces:

- Streaming data pipelines
- Apache Kafka
- Event-driven processing
- Real-time ML inference
- Streaming analytics

2. Workshop Goal

Design and implement a streaming ETL pipeline capable of generating real-time predictions using Apache Kafka and a pre-trained machine learning model.

3. Learning Objectives

By the end of this workshop, students will be able to:

1. Integrate heterogeneous datasets into a unified analytical schema.
2. Build a batch ETL pipeline for machine learning preparation.
3. Train and serialize a regression model.
4. Implement a Kafka producer that streams events.
5. Implement a Kafka consumer that performs real-time inference.
6. Define and validate a basic event contract before prediction.

7. Store prediction results in a database.
8. Build analytical visualizations using prediction results.

4. General Architecture

Offline Process

Historical CSV Files
↓
Data Profiling (EDA + Cleaning + Schema Harmonization)
↓
Feature Engineering
↓
Train Regression Model
↓
Save preprocessing + model pipeline (model.pkl)

Streaming Process

Historical CSV Files
↓
Kafka Producer
(stream raw data)
↓
Kafka Topic
↓
Kafka Consumer
↓
Store raw event
↓
Validate Event Contract
↓
Prepare Features Using the Training Pipeline
↓
↓
Load model.pkl
↓
Generate Prediction
↓
Store Prediction Results
↓
Dashboard & KPIs

5. Dataset Description

You are provided with multiple CSV files containing World Happiness data from different years.

Files:

- 2015.csv
- 2016.csv
- 2017.csv
- 2018.csv
- 2019.csv

The datasets contain:

- Happiness score
- GDP
- Health indicators
- Family/social support
- Freedom indicators
- Corruption perception
- Generosity
- Country information

Important:
The datasets do NOT share exactly the same schema. You must analyze and harmonize the datasets before integrating them.
Streaming simulation note: the CSV files are bounded historical sources. In Part B, the producer will read records one by one and publish them as events to simulate data arriving over time.

6. Activities

PART A — Data Profiling and Machine Learning

Objective: Build a batch ETL pipeline capable of preparing data for machine learning.

Step 1 — Exploratory Data Analysis (EDA)
Perform EDA on all datasets.

You must analyze:

- Missing values
- Duplicated records

- Inconsistent column names
- Inconsistent data types
- Schema differences between years
- Potential outliers

Deliverables:

- EDA notebook
- Data quality observations
- Unified schema proposal

Step 2 — Data Cleaning and Harmonization

Design a unified analytical schema.

You must:

- Standardize column names
- Standardize data types
- Remove or handle missing values
- Resolve schema inconsistencies
- Merge datasets into a unified dataset

Important:

You must justify your cleaning decisions.

Step 3 — Feature Engineering

Prepare features for machine learning.

Generate descriptive statistics and visualizations to explore relationships among variables (e.g., GDP, social support, life expectancy).

Select and preprocess the features that are most relevant for predicting the happiness score.

Requirements:

- Select meaningful features
- Justify feature selection
- Avoid target leakage
- Handle categorical data if necessary
- Normalize or scale features if required

Important:

The focus of this workshop is pipeline integration, not model optimization. Use a simple regression model.

If your model requires learned preprocessing (for example, scaling or encoding), serialize the preprocessing together with the model using a reproducible scikit-learn Pipeline.

Step 4 — Train Regression Model
Train a regression model capable of predicting happiness score.

Suggested models:
- Linear Regression
- Random Forest Regressor
- Decision Tree Regressor

You must:
- Split the data into training and testing sets. Suggestion:
  - **70% training data**
  - **30% testing data**
- Train the model
- Evaluate the model
- Save the trained model as:
  - model.pkl

Suggested metrics:
- MAE
- RMSE
- R^2

Deliverables:
- Training notebook
- Evaluation metrics
- Serialized model

PART B — Streaming ETL with Apache Kafka

Objective
Implement a streaming inference pipeline using Kafka.

Kafka Requirements

You must use:
- Apache Kafka
- Python producer
- Python consumer

Recommended environment:

- Docker Compose

Kafka Architecture

Producer
↓
Kafka Topic
↓
Consumer

Step 5 — Kafka Producer

Implement a producer that reads the prepared historical data record by record and publishes each record as a simulated streaming event.

Requirements:

- Stream records one by one
- Serialize events as JSON
- Send events to a Kafka topic using country as the event key
- Simulate event arrival over time (for example, a short configurable delay between records)
- Use a topic with 3 partitions and replication factor 1 for the local single-broker environment

Required topic name: happiness-events

Required JSON format:

```
{
"event_id": "E001",
"country": "Colombia",
"year": 2019,
"gdp": 1.2,
"family": 0.8,
"health": 0.9,
"freedom": 0.6,
"generosity": 0.3,
"corruption": 0.1,
"actual_happiness_score": 6.2,
"event_time": "2026-10-30T15:30:00"
}
```

Event Contract (minimum requirements)

Before implementing the consumer, define the contract expected by downstream processing. At minimum:

- event_id: non-empty string; unique identifier for traceability.
- country: non-empty string; also used as the Kafka event key.
- year: integer corresponding to the historical observation.
- gdp, family, health, freedom, generosity, corruption: numeric values compatible with the unified training schema.
- actual_happiness_score: numeric value used only for evaluation; it must NOT be used as a model input feature.
- event_time: valid ISO-8601 timestamp representing when the producer emitted the simulated event.

Important: Kafka transports the record, but the application is responsible for interpreting and validating the business fields.

Step 6 — Kafka Consumer

Implement a consumer that performs real-time inference.

The consumer must:

- Use consumer group: happiness-ml-team

1. Receive streaming events
2. Store the raw incoming event in a database table before applying any transformation or prediction.
3. Validate the incoming event against the defined event contract
4. Apply the same feature ordering and preprocessing used during model training
5. Load the serialized model
6. Generate predictions
7. Store prediction results in a database

Raw Event Storage Requirement

Before validation or prediction, the consumer must load the original Kafka message into a raw table. Suggested table: raw_happiness_events

The raw table preserves the original incoming event exactly as it arrived from Kafka. This supports traceability, auditing, debugging, and future reprocessing if transformation or prediction rules change.

Invalid records should also be stored in the raw table, but they must be marked with an appropriate processing status, such as:

- VALID
- INVALID_SCHEMA

- INVALID_VALUES
- PREDICTION_ERROR

Important — Invalid records must:
- be stored in the raw table
- be skipped from prediction
- NOT crash the pipeline

Event Contract and Validation Requirements

You must validate:
- Required fields defined by the event contract
- Missing fields
- Invalid data types
- Invalid numerical values
- Missing features required by the model

PART C — Prediction Storage and Analytics

Objective
Store predictions and generate analytical insights.

Step 7 — Database Design
Design a small analytical model for predictions.

Minimum required tables:

Raw Table:
raw_happiness_events
This table stores the original Kafka events exactly as they were received but they must be marked with an appropriate processing status

Fact Table:
fact_predictions

Suggested columns:
- prediction_id
- raw_event_id
- country_id
- date_id
- actual_score
- predicted_score
- prediction_error

- prediction_timestamp

Dimensions:
Minimum required dimensions:

- dim_country
- dim_date

The raw_happiness_events table is a raw/staging table for traceability, not a dimension.
You may add additional analytical dimensions if justified.

Step 8 — Load Raw Events and Prediction Results

The consumer must insert both raw events and prediction results into the database.

Requirements:

- Store the original Kafka message in raw_happiness_events.
- Store actual score.
- Store predicted score.
- Store prediction error.
- Store event_time and prediction_timestamp.
- Link each prediction to the original raw event using raw_event_id.

This design allows the prediction result to be traced back to the exact event that produced it.

Suggested databases:

- PostgreSQL (recommended) or SQLite for a local implementation

Step 9 — Dashboard and KPIs
Build a dashboard connected to your prediction database.

Minimum required KPIs:

1. Average prediction error
2. Predictions by country
3. Predicted vs actual score
4. Prediction trends over time

Suggested tools:

- Power BI
- Looker Studio
- Tableau

Important:

The dashboard must query the database, NOT CSV files.

7. Technical Requirements

- Python
- Apache Kafka
- Pandas
- Scikit-learn
- SQL database

8. Recommended Folder Structure

```
project/
│
├── data/
│   ├── raw/
│   ├── processed/
│   └── streaming/
│
├── notebooks/
│   ├── eda.ipynb
│   └── model_training.ipynb
│
├── kafka/
│   ├── producer.py
│   └── consumer.py
│
├── models/
│   └── model.pkl
│
├── sql/
│   ├── create_tables.sql
│   └── kpis.sql
│
├── dashboards/
│
├── docker-compose.yml
│
├── requirements.txt
│
└── README.md
```

9. Deliverables

1. GitHub Repository

The repository must include:

- ETL notebooks
- Producer and consumer scripts
- Serialized model
- SQL scripts
- Dashboard files/screenshots
- requirements.txt
- README.md

2. README.md (MANDATORY)

The README must include:

- Project description
- Architecture explanation
- Data cleaning decisions
- Feature engineering decisions
- Kafka pipeline and event contract explanation
- Database schema
- Dashboard explanation
- Execution instructions

3. Dashboard

Include:

- screenshots
- dashboard file or link
- KPI explanations

10. Evaluation Criteria

Project

Criteria	Weight
Data Integration & Cleaning	1.0
Feature Engineering	0.5
ML Pipeline	0.5
Kafka Producer	0.5
Kafka Consumer	0.5
Event Contract & Validation	0.5
Database Design & Loading	0.5

Dashboard & KPIs	0.5
Documentation & Reproducibility	0.5

- Project: 70%
- Presentation (Clarity and Structure, Communication and Professionalism): 30%

Key Insight

This workshop is NOT focused on maximizing ML accuracy.

The main goal is:

- Building an integrated streaming ETL pipeline
- capable of generating real-time predictions.

Focus on:

- clean architecture
- reproducibility
- pipeline reliability
- data consistency
- streaming integration rather than model complexity.